```
================================================================
INTERVIEW PREP — PART 2 (EXTENDED)
C/C++ Theory + DSA + Sorting + Searching + Matrix + MCQ Tricks
Network Marvels | Software Engineer | Thane
Samaranjan Manjhi
================================================================


================================================================
PART A — C/C++ DEEP THEORY QUESTIONS
================================================================

------------------------------------------------------------
A1. MEMORY MANAGEMENT
------------------------------------------------------------

Q. Difference between malloc/calloc/realloc/free vs new/delete?
A. malloc:   allocates raw bytes, returns void*, no constructor called
   calloc:   like malloc but zero-initializes memory
   realloc:  resize existing allocation
   free:     deallocates malloc/calloc memory
   new:      allocates + calls constructor, returns typed pointer
   delete:   calls destructor + deallocates
   new[]:    allocates array + calls constructors for each element
   delete[]: must match new[] — calls destructor for each element

   TRICK: Always match malloc/free and new/delete. Never mix.
   Mixing (free on new'd ptr) = undefined behavior.

Q. What is a memory leak? How do you detect it?
A. Memory leak: heap memory allocated but never freed. Program's
   memory usage grows until crash or OOM.
   Detection: Valgrind (--leak-check=full), AddressSanitizer (-fsanitize=address)
   YOUR ANSWER: "I ran Valgrind on my DLP scanner daemon — found a
   leak in the Poppler PDF handler where I forgot to delete the
   document object on error paths. RAII wrapper fixed it."

Q. What is dangling pointer? How to avoid?
A. Dangling pointer: pointer to memory that has been freed.
   Accessing it = undefined behavior (crash or corruption).
   Fix: set pointer to nullptr after free. Use smart pointers.
   int* p = new int(5);
   delete p;
   p = nullptr;  // good practice
   *p = 10;      // crash if p not set to nullptr

Q. What is stack overflow?
A. Stack memory is limited (~1-8 MB). Deep recursion fills the stack.
   Each function call pushes a stack frame (local vars + return addr).
   When stack is exhausted → stack overflow → segfault.
   Fix: use iterative approach or increase stack size.
```

```
Q. new vs new(nothrow)?
A. new throws std::bad_alloc on failure.
   new(nothrow) returns nullptr on failure — use when you want to
   check instead of catching exception.
   int* p = new(nothrow) int[1000000];
   if(!p) { /* handle gracefully */ }

Q. What is memory alignment? Why does it matter?
A. CPU reads memory efficiently when data is aligned to its size boundary.
   int must be at 4-byte boundary, double at 8-byte boundary.
   Compiler adds padding to structs to enforce alignment.
   Misaligned access: UB on x86, crash on ARM.

Q. What is the heap vs BSS vs data vs text segment?
A. text:  code (instructions), read-only
   data:  initialized global/static variables
   BSS:   uninitialized global/static (zero-initialized at start)
   heap:  dynamic allocation (malloc/new), grows upward
   stack: local variables, grows downward
   Each process gets these segments via the OS loader (execve).


------------------------------------------------------------
A2. POINTERS & REFERENCES
------------------------------------------------------------

Q. Pointer vs reference in C++?
A. Pointer: can be null, can be reassigned, needs * to dereference
   Reference: must be initialized at declaration, cannot be null,
              cannot be reassigned, accessed like the original variable
   Use reference when: passing to function without copy, guaranteed non-null
   Use pointer when: optional (nullable), need to change what it points to

Q. const correctness — 4 forms of const with pointer?
A. int x = 5;
   int* p = &x;          // non-const ptr to non-const int
   const int* p = &x;    // non-const ptr to const int (data is const)
   int* const p = &x;    // const ptr to non-const int (ptr is const)
   const int* const p = &x; // both const

   TRICK: read right-to-left from pointer name.
   "const int* p" = p is a pointer to const int.
   "int* const p" = p is a const pointer to int.

Q. What is a void pointer? When to use?
A. void*: generic pointer, no type info, cannot be dereferenced directly.
   Must cast before use. Used in C-style generic functions (malloc returns void*).
   int x = 5;
   void* p = &x;
   int* ip = (int*)p;  // must cast
```

```
   printf("%d", *ip);  // now valid

Q. Function pointers in C — syntax?
A. int (*fp)(int, int);  // fp is ptr to function taking 2 ints, returning int
   int add(int a, int b) { return a+b; }
   fp = add;
   int result = fp(3, 4);  // calls add(3, 4)
   In C++: use std::function<int(int,int)> instead — safer.

Q. What is a double pointer? When used?
A. int x=5, *p=&x, **pp=&p;
   **pp = 10;  // modifies x through two levels
   Use: passing pointer by pointer (to modify what ptr points to),
   dynamic 2D arrays, argv (char**).


------------------------------------------------------------
A3. TEMPLATES & STL
------------------------------------------------------------

Q. What is a template? Function template vs class template?
A. Template: code that works for any type — compile-time polymorphism.
   template<typename T>
   T maxVal(T a, T b) { return (a > b) ? a : b; }
   // maxVal(3,5) → int version compiled
   // maxVal(3.0,5.0) → double version compiled
   Class template: template<typename T> class Stack { ... };

Q. Template specialization?
A. Provide a specific implementation for a particular type.
   template<> void print<bool>(bool val) {
       cout << (val ? "true" : "false");
   }

Q. STL containers — which to use when?
A. vector:       dynamic array, random access O(1), insert at end O(1) amortized
   list:         doubly linked list, O(1) insert anywhere, no random access
   deque:        double-ended queue, O(1) front/back insert
   set:          sorted unique elements, O(log n) all ops (BST internally)
   unordered_set: hash table, O(1) avg lookup, no order
   map:          sorted key-value, O(log n) (BST internally)
   unordered_map: hash table key-value, O(1) avg
   stack:        LIFO, adaptor over deque
   queue:        FIFO, adaptor over deque
   priority_queue: max-heap by default, O(log n) push/pop

Q. vector vs array?
A. array: fixed size, stack-allocated (small arrays), no overhead
   vector: dynamic size, heap-allocated, slight overhead
   vector.size() = current elements. vector.capacity() = allocated space.
   push_back doubles capacity when full (amortized O(1)).
```

```
Q. Iterator invalidation — when does it happen?
A. vector: ALL iterators invalid after push_back (if reallocation), or after
           erase (iterators at/after erased element invalid)
   list:   only the erased element's iterator invalid
   map:    only the erased element's iterator invalid
   RULE: don't cache iterators across modifying operations on vector.

Q. std::sort vs std::stable_sort?
A. sort: O(n log n), not stable (equal elements may reorder), uses introsort
   stable_sort: O(n log n) with O(n) extra space, preserves relative order
                of equal elements
   For structs: provide comparator:
   sort(v.begin(), v.end(), [](const Person& a, const Person& b){
       return a.age < b.age;
   });

Q. Lambda syntax in C++11?
A. [capture](params) -> return_type { body }
   auto add = [](int a, int b) { return a+b; };
   int x=5;
   auto addX = [x](int a) { return a+x; };  // capture x by value
   auto addXRef = [&x](int a) { x++; return a+x; }; // by reference


------------------------------------------------------------
A4. INHERITANCE & POLYMORPHISM (DEEPER)
------------------------------------------------------------

Q. Types of inheritance in C++?
A. Public inheritance:    IS-A relationship. public→public, protected→protected
   Protected inheritance: implementation inheritance. public→protected
   Private inheritance:   implementation inheritance. public→private
   Most common: public inheritance (IS-A).

Q. What is object slicing?
A. When a Derived object is assigned to a Base object (by value),
   the derived part is "sliced off" — only base members copied.
   Derived d; Base b = d;  // d's extra data LOST
   Fix: use pointers/references to base class (polymorphism).

Q. Abstract class vs interface in C++?
A. C++ has no "interface" keyword (unlike Java).
   Abstract class: has at least one pure virtual function.
   Interface (C++ idiom): class with ALL pure virtual functions, no data.
   class IScanner {
       virtual bool scan(const string& path) = 0;  // pure virtual
       virtual ~IScanner() = default;
   };

Q. When to use virtual destructor?
```

```
A. ALWAYS when a class has virtual functions (i.e., will be used polymorphically).
   Without virtual destructor: deleting Derived via Base* only calls ~Base().
   Derived resources NOT freed → memory leak.
   class Base { virtual ~Base() {} };  // always add this

Q. What is covariant return type?
A. Derived class override can return a pointer/reference to a more derived type.
   class Base { virtual Base* clone() { return new Base(*this); } };
   class Derived : public Base { virtual Derived* clone() { return new Derived(*this); } };
   Both are valid overrides — compiler accepts the covariant return.

Q. Difference between overloading, overriding, and hiding?
A. Overloading: same name, different params, SAME class (compile-time)
   Overriding:  same name, same params, DERIVED class, virtual (runtime)
   Hiding:      same name in derived class WITHOUT virtual — hides base version


------------------------------------------------------------
A5. EXCEPTION HANDLING
------------------------------------------------------------

Q. try/catch/throw syntax?
A. try {
       if(error) throw runtime_error("something failed");
   }
   catch(const runtime_error& e) { cerr << e.what(); }
   catch(const exception& e)     { cerr << e.what(); }
   catch(...)                    { cerr << "unknown exception"; }
   Order: most specific exception first (most derived first).

Q. What is exception safety? 3 levels?
A. No-throw guarantee:  function never throws (noexcept)
   Strong guarantee:    if exception thrown, state unchanged (rollback)
   Basic guarantee:     object in valid but unspecified state after exception
   Add noexcept to move constructors for STL performance.

Q. What is noexcept?
A. Specifies function won't throw. Enables compiler optimizations.
   void cleanup() noexcept { /* guaranteed no throw */ }
   STL uses it: if move constructor is noexcept, vector uses move on resize.
   If noexcept function DOES throw: std::terminate() is called immediately.

Q. RAII and exceptions — why RAII matters?
A. If exception thrown mid-function, stack unwinds — destructors called.
   Raw pointers NOT freed (no destructor). RAII wrappers ARE freed.
   This is why smart pointers beat raw pointers in exception-heavy code.


------------------------------------------------------------
A6. CONCURRENCY DEEP (for your daemon expertise)
------------------------------------------------------------
```

```
Q. std::atomic — when to use instead of mutex?
A. For simple types (int, bool, pointer): std::atomic gives lock-free
   thread-safe operations. Faster than mutex for single variable.
   std::atomic<int> counter(0);
   counter++;  // thread-safe, no mutex needed

Q. What is lock_guard vs unique_lock?
A. lock_guard:  simpler, RAII lock, cannot unlock manually
   unique_lock: more flexible, can unlock/relock, required for condition_variable
   Use lock_guard for simple critical sections.
   Use unique_lock when calling condition_variable.wait().

Q. What is memory ordering in atomic operations?
A. memory_order_relaxed: no ordering guarantees, just atomicity
   memory_order_acquire: reads after this see all writes before release
   memory_order_release: writes before this visible after matching acquire
   memory_order_seq_cst: total order — default, safest, slowest
   For most uses: default (seq_cst) is fine.

Q. Thread pool — why use one?
A. Creating/destroying threads is expensive (OS syscall).
   Thread pool: create N threads once, reuse them via a task queue.
   In your daemon: scan worker threads pick jobs from the Unix socket queue.

Q. What is a spinlock vs mutex?
A. Spinlock: thread burns CPU in a tight loop waiting. Good for very short
             waits (microseconds). Bad if wait is long.
   Mutex:    thread sleeps and is woken by OS. No CPU waste during wait.
             Context switch overhead. Better for longer waits.


------------------------------------------------------------
A7. C++ MOVE SEMANTICS (frequently asked at Sr level)
------------------------------------------------------------

Q. What is move semantics? Why introduced in C++11?
A. Before C++11: returning a vector from a function copied ALL elements.
   Move: transfer ownership of resources instead of copying.
   string a = "hello";
   string b = std::move(a);  // a is now empty, b owns the data
   No copy made — just pointer swap. O(1) instead of O(n).

Q. lvalue vs rvalue?
A. lvalue: has a name, has an address, can be on left of assignment
   rvalue: temporary, no name, will be destroyed after expression
   int x = 5;   // x is lvalue, 5 is rvalue
   string s = string("temp");  // string("temp") is rvalue

Q. Move constructor vs copy constructor?
A. Copy constructor: takes const T&, makes a deep copy
```

```
   Move constructor: takes T&&, steals the resources (sets source to null)
   class MyVec {
       int* data; int size;
       MyVec(MyVec&& other)  // move constructor
           : data(other.data), size(other.size) {
           other.data = nullptr;  // source now owns nothing
       }
   };

Q. When are move semantics called automatically?
A. When returning a local variable from function (NRVO/RVO).
   When inserting rvalue into container.
   Explicitly with std::move().
   TRICK: std::move doesn't actually move — it just casts to rvalue ref.
   The move constructor/assignment does the actual moving.


================================================================
PART B — SORTING ALGORITHMS (complete)
================================================================

------------------------------------------------------------
B1. BUBBLE SORT
------------------------------------------------------------
Idea: repeatedly swap adjacent elements if out of order.
      Largest element "bubbles" to end each pass.

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for(int i = 0; i < n-1; i++) {
        bool swapped = false;
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
        if(!swapped) break;  // already sorted — early exit
    }
}

Time:  Best O(n) [already sorted], Average O(n²), Worst O(n²)
Space: O(1) — in-place
Stable: YES
Use when: teaching only. Never in production.


------------------------------------------------------------
B2. SELECTION SORT
------------------------------------------------------------
Idea: find minimum element, put it in correct position. Repeat.
```

```
void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for(int i = 0; i < n-1; i++) {
        int minIdx = i;
        for(int j = i+1; j < n; j++)
            if(arr[j] < arr[minIdx]) minIdx = j;
        swap(arr[i], arr[minIdx]);
    }
}

Time:  O(n²) always (no early exit)
Space: O(1)
Stable: NO (swap can change relative order)
Use when: minimizing number of swaps matters (each pass does at most 1 swap).


------------------------------------------------------------
B3. INSERTION SORT
------------------------------------------------------------
Idea: like sorting playing cards. Take one element, insert into
      correct position among already-sorted left portion.

void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for(int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i-1;
        while(j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}

Time:  Best O(n) [sorted input], Average O(n²), Worst O(n²)
Space: O(1)
Stable: YES
Use when: small arrays (< 20 elements), nearly sorted data.
          std::sort uses insertion sort for small sub-arrays (introsort).


------------------------------------------------------------
B4. MERGE SORT
------------------------------------------------------------
Idea: divide array in half, sort each half, merge sorted halves.
      Classic divide and conquer.

void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> left(arr.begin()+l, arr.begin()+m+1);
```

```
    vector<int> right(arr.begin()+m+1, arr.begin()+r+1);
    int i=0, j=0, k=l;
    while(i<left.size() && j<right.size())
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while(i<left.size()) arr[k++] = left[i++];
    while(j<right.size()) arr[k++] = right[j++];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if(l >= r) return;
    int m = l + (r-l)/2;
    mergeSort(arr, l, m);
    mergeSort(arr, m+1, r);
    merge(arr, l, m, r);
}
// Call: mergeSort(arr, 0, arr.size()-1);

Time:  O(n log n) always
Space: O(n) — extra array for merging
Stable: YES
Use when: need guaranteed O(n log n), linked lists (no random access),
          external sort (data doesn't fit in RAM).


------------------------------------------------------------
B5. QUICK SORT
------------------------------------------------------------
Idea: pick a pivot, partition array so left < pivot < right.
      Recursively sort left and right.

int partition(vector<int>& arr, int lo, int hi) {
    int pivot = arr[hi];
    int i = lo - 1;
    for(int j = lo; j < hi; j++) {
        if(arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[hi]);
    return i+1;
}

void quickSort(vector<int>& arr, int lo, int hi) {
    if(lo < hi) {
        int pi = partition(arr, lo, hi);
        quickSort(arr, lo, pi-1);
        quickSort(arr, pi+1, hi);
    }
}
// Call: quickSort(arr, 0, arr.size()-1);
```

```
Time:  Best/Average O(n log n), Worst O(n²) [sorted input, bad pivot]
Space: O(log n) stack space
Stable: NO
Use when: general purpose (std::sort is introsort based on quicksort).
Trick: use random pivot to avoid worst case.


------------------------------------------------------------
B6. HEAP SORT
------------------------------------------------------------
Idea: build a max-heap, repeatedly extract max to sort.

void heapify(vector<int>& arr, int n, int i) {
    int largest = i, left = 2*i+1, right = 2*i+2;
    if(left < n && arr[left] > arr[largest]) largest = left;
    if(right < n && arr[right] > arr[largest]) largest = right;
    if(largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int>& arr) {
    int n = arr.size();
    for(int i = n/2-1; i >= 0; i--) heapify(arr, n, i); // build heap
    for(int i = n-1; i > 0; i--) {
        swap(arr[0], arr[i]);      // move max to end
        heapify(arr, i, 0);        // re-heapify
    }
}

Time:  O(n log n) always
Space: O(1) — in-place
Stable: NO
Use when: need O(n log n) guaranteed + O(1) space.


------------------------------------------------------------
B7. COUNTING SORT (for integers in known range)
------------------------------------------------------------
Idea: count frequency of each value, reconstruct sorted array.

void countingSort(vector<int>& arr, int maxVal) {
    vector<int> count(maxVal+1, 0);
    for(int x : arr) count[x]++;
    int idx = 0;
    for(int i = 0; i <= maxVal; i++)
        while(count[i]--) arr[idx++] = i;
}

```

```
Time:  O(n + k) where k = range of values
Space: O(k)
Stable: YES (with extra work)
Use when: integers in small range (e.g. sort grades 0-100).


------------------------------------------------------------
B8. SORTING QUICK REFERENCE TABLE
------------------------------------------------------------
Algorithm     | Best    | Average | Worst   | Space | Stable
Bubble        | O(n)    | O(n²)   | O(n²)   | O(1)  | YES
Selection     | O(n²)   | O(n²)   | O(n²)   | O(1)  | NO
Insertion     | O(n)    | O(n²)   | O(n²)   | O(1)  | YES
Merge         | O(nlogn)| O(nlogn)| O(nlogn)| O(n)  | YES
Quick         | O(nlogn)| O(nlogn)| O(n²)   | O(logn)| NO
Heap          | O(nlogn)| O(nlogn)| O(nlogn)| O(1)  | NO
Counting      | O(n+k)  | O(n+k)  | O(n+k)  | O(k)  | YES

INTERVIEWER TRICK QUESTION: "Which is better, merge sort or quick sort?"
ANSWER: "In practice, quick sort is faster due to better cache performance
and lower constant factors — even though both are O(n log n). Merge sort
is preferred when stability is required or for external sort.
std::sort uses introsort — a hybrid of quicksort, heapsort, and insertion sort."


================================================================
PART C — SEARCHING ALGORITHMS
================================================================

------------------------------------------------------------
C1. LINEAR SEARCH
------------------------------------------------------------
int linearSearch(vector<int>& arr, int target) {
    for(int i = 0; i < arr.size(); i++)
        if(arr[i] == target) return i;
    return -1;
}
Time: O(n). Use when: unsorted array, small array.


------------------------------------------------------------
C2. BINARY SEARCH (sorted array — most important)
------------------------------------------------------------
int binarySearch(vector<int>& arr, int target) {
    int lo = 0, hi = arr.size()-1;
    while(lo <= hi) {
        int mid = lo + (lo+hi)/2;  // avoid overflow: lo + (hi-lo)/2
        if(arr[mid] == target) return mid;
        else if(arr[mid] < target) lo = mid+1;
        else hi = mid-1;
    }
```

```
    return -1;
}
Time: O(log n). PREREQUISITE: array must be sorted.

COMMON VARIATIONS:
// Find first occurrence of target
int firstOccurrence(vector<int>& arr, int target) {
    int lo=0, hi=arr.size()-1, result=-1;
    while(lo<=hi) {
        int mid=lo+(hi-lo)/2;
        if(arr[mid]==target) { result=mid; hi=mid-1; } // don't stop, go left
        else if(arr[mid]<target) lo=mid+1;
        else hi=mid-1;
    }
    return result;
}

// Find last occurrence
int lastOccurrence(vector<int>& arr, int target) {
    int lo=0, hi=arr.size()-1, result=-1;
    while(lo<=hi) {
        int mid=lo+(hi-lo)/2;
        if(arr[mid]==target) { result=mid; lo=mid+1; } // go right
        else if(arr[mid]<target) lo=mid+1;
        else hi=mid-1;
    }
    return result;
}

// Lower bound: first index where arr[i] >= target
int lowerBound(vector<int>& arr, int target) {
    return lower_bound(arr.begin(), arr.end(), target) - arr.begin();
}

// Upper bound: first index where arr[i] > target
int upperBound(vector<int>& arr, int target) {
    return upper_bound(arr.begin(), arr.end(), target) - arr.begin();
}


------------------------------------------------------------
C3. BINARY SEARCH ON ROTATED SORTED ARRAY
------------------------------------------------------------
// Array was sorted then rotated: [4,5,6,7,0,1,2]
int searchRotated(vector<int>& nums, int target) {
    int lo=0, hi=nums.size()-1;
    while(lo<=hi) {
        int mid=lo+(hi-lo)/2;
        if(nums[mid]==target) return mid;
        if(nums[lo]<=nums[mid]) {         // left half is sorted
            if(nums[lo]<=target && target<nums[mid]) hi=mid-1;
```

```
            else lo=mid+1;
        } else {                          // right half is sorted
            if(nums[mid]<target && target<=nums[hi]) lo=mid+1;
            else hi=mid-1;
        }
    }
    return -1;
}


------------------------------------------------------------
C4. SEARCH IN 2D MATRIX (sorted rows and columns)
------------------------------------------------------------
// Each row sorted. Last elem of row < first elem of next row.
bool searchMatrix(vector<vector<int>>& mat, int target) {
    int m=mat.size(), n=mat[0].size();
    int lo=0, hi=m*n-1;
    while(lo<=hi) {
        int mid=lo+(hi-lo)/2;
        int val=mat[mid/n][mid%n];   // convert 1D index to 2D
        if(val==target) return true;
        else if(val<target) lo=mid+1;
        else hi=mid-1;
    }
    return false;
}

// Variation: rows sorted, columns sorted (but rows not sequential)
// Start from top-right corner:
bool searchMatrix2(vector<vector<int>>& mat, int target) {
    int r=0, c=mat[0].size()-1;
    while(r<mat.size() && c>=0) {
        if(mat[r][c]==target) return true;
        else if(mat[r][c]>target) c--;
        else r++;
    }
    return false;
}
Time: O(m+n) — much better than O(m*n)


------------------------------------------------------------
C5. JUMP SEARCH
------------------------------------------------------------
// For sorted arrays — jump by sqrt(n) steps, then linear search
int jumpSearch(vector<int>& arr, int target) {
    int n=arr.size(), step=sqrt(n), prev=0;
    while(arr[min(step,n)-1] < target) {
        prev=step; step+=sqrt(n);
        if(prev>=n) return -1;
    }
```

```
    while(arr[prev] < target) {
        prev++;
        if(prev==min(step,n)) return -1;
    }
    return (arr[prev]==target) ? prev : -1;
}
Time: O(sqrt(n)). Between linear and binary search.


================================================================
PART D — ARRAY MANIPULATION PROBLEMS
================================================================

------------------------------------------------------------
D1. ROTATE ARRAY — RIGHT BY K
------------------------------------------------------------
// Method 1: using extra space
void rotateRight(vector<int>& arr, int k) {
    int n = arr.size();
    k = k % n;  // handle k > n
    vector<int> temp(arr.end()-k, arr.end()); // last k elements
    temp.insert(temp.end(), arr.begin(), arr.end()-k);
    arr = temp;
}

// Method 2: reverse trick (O(1) space) — PREFERRED
void rotateRight(vector<int>& arr, int k) {
    int n = arr.size();
    k = k % n;
    reverse(arr.begin(), arr.end());         // reverse all
    reverse(arr.begin(), arr.begin()+k);     // reverse first k
    reverse(arr.begin()+k, arr.end());       // reverse rest
}
// [1,2,3,4,5] rotate right by 2 → [4,5,1,2,3]
// Step1: [5,4,3,2,1]
// Step2: [4,5,3,2,1]
// Step3: [4,5,1,2,3] ✓
Time: O(n), Space: O(1)


------------------------------------------------------------
D2. ROTATE ARRAY — LEFT BY K
------------------------------------------------------------
void rotateLeft(vector<int>& arr, int k) {
    int n = arr.size();
    k = k % n;
    reverse(arr.begin(), arr.begin()+k);     // reverse first k
    reverse(arr.begin()+k, arr.end());       // reverse rest
    reverse(arr.begin(), arr.end());         // reverse all
}
// [1,2,3,4,5] rotate left by 2 → [3,4,5,1,2]
```

```
// Or: just call rotateRight(arr, n-k)


------------------------------------------------------------
D3. KADANE'S ALGORITHM — MAXIMUM SUBARRAY SUM
------------------------------------------------------------
int maxSubArray(vector<int>& nums) {
    int maxSum = nums[0], currSum = nums[0];
    for(int i = 1; i < nums.size(); i++) {
        currSum = max(nums[i], currSum + nums[i]);
        maxSum = max(maxSum, currSum);
    }
    return maxSum;
}
// [-2,1,-3,4,-1,2,1,-5,4] → 6 ([4,-1,2,1])
Time: O(n), Space: O(1)


------------------------------------------------------------
D4. FIND MISSING NUMBER IN ARRAY [1..N]
------------------------------------------------------------
int missingNumber(vector<int>& nums) {
    int n = nums.size();
    int expected = n*(n+1)/2;
    int actual = 0;
    for(int x : nums) actual += x;
    return expected - actual;
}
// XOR method (avoids overflow for large n):
int missingNumber(vector<int>& nums) {
    int xorAll = 0;
    for(int i = 0; i <= nums.size(); i++) xorAll ^= i;
    for(int x : nums) xorAll ^= x;
    return xorAll;
}


------------------------------------------------------------
D5. FIND DUPLICATE IN ARRAY (no extra space, no modification)
------------------------------------------------------------
// Floyd's cycle detection on array values as pointers
int findDuplicate(vector<int>& nums) {
    int slow = nums[0], fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while(slow != fast);
    slow = nums[0];
    while(slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
```

```
    }
    return slow;
}
// [1,3,4,2,2] → 2


------------------------------------------------------------
D6. MOVE ZEROS TO END
------------------------------------------------------------
void moveZeroes(vector<int>& nums) {
    int writeIdx = 0;
    for(int i = 0; i < nums.size(); i++)
        if(nums[i] != 0) nums[writeIdx++] = nums[i];
    while(writeIdx < nums.size()) nums[writeIdx++] = 0;
}
// [0,1,0,3,12] → [1,3,12,0,0]


------------------------------------------------------------
D7. MERGE SORTED ARRAYS (two sorted arrays into one)
------------------------------------------------------------
vector<int> mergeSorted(vector<int>& a, vector<int>& b) {
    vector<int> res;
    int i=0, j=0;
    while(i<a.size() && j<b.size())
        res.push_back(a[i]<=b[j] ? a[i++] : b[j++]);
    while(i<a.size()) res.push_back(a[i++]);
    while(j<b.size()) res.push_back(b[j++]);
    return res;
}


------------------------------------------------------------
D8. STOCK BUY SELL — BEST PROFIT (one transaction)
------------------------------------------------------------
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for(int p : prices) {
        minPrice = min(minPrice, p);
        maxProfit = max(maxProfit, p - minPrice);
    }
    return maxProfit;
}
// [7,1,5,3,6,4] → 5 (buy at 1, sell at 6)


------------------------------------------------------------
D9. TRAPPING RAIN WATER
------------------------------------------------------------
int trap(vector<int>& height) {
    int lo=0, hi=height.size()-1;
```

```
    int leftMax=0, rightMax=0, water=0;
    while(lo<hi) {
        if(height[lo]<height[hi]) {
            if(height[lo]>=leftMax) leftMax=height[lo];
            else water+=leftMax-height[lo];
            lo++;
        } else {
            if(height[hi]>=rightMax) rightMax=height[hi];
            else water+=rightMax-height[hi];
            hi--;
        }
    }
    return water;
}
Time: O(n), Space: O(1)


------------------------------------------------------------
D10. SLIDING WINDOW — MAXIMUM SUM SUBARRAY OF SIZE K
------------------------------------------------------------
int maxSumSubarrayK(vector<int>& arr, int k) {
    int windowSum=0, maxSum=INT_MIN;
    for(int i=0; i<arr.size(); i++) {
        windowSum += arr[i];
        if(i >= k-1) {
            maxSum = max(maxSum, windowSum);
            windowSum -= arr[i-k+1];
        }
    }
    return maxSum;
}
// [2,1,5,1,3,2], k=3 → 9 ([5,1,3])


------------------------------------------------------------
D11. SUBARRAY SUM EQUALS K (prefix sum + hashmap)
------------------------------------------------------------
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int,int> prefixCount;
    prefixCount[0] = 1;
    int sum=0, count=0;
    for(int x : nums) {
        sum += x;
        if(prefixCount.count(sum-k)) count += prefixCount[sum-k];
        prefixCount[sum]++;
    }
    return count;
}


------------------------------------------------------------
```

```
D12. PRODUCT OF ARRAY EXCEPT SELF (no division)
------------------------------------------------------------
vector<int> productExceptSelf(vector<int>& nums) {
    int n=nums.size();
    vector<int> res(n,1);
    // left pass: res[i] = product of all elements left of i
    int left=1;
    for(int i=0; i<n; i++) { res[i]=left; left*=nums[i]; }
    // right pass: multiply by product of all elements right of i
    int right=1;
    for(int i=n-1; i>=0; i--) { res[i]*=right; right*=nums[i]; }
    return res;
}
// [1,2,3,4] → [24,12,8,6]


================================================================
PART E — MATRIX PROBLEMS
================================================================

------------------------------------------------------------
E1. ROTATE MATRIX 90 DEGREES CLOCKWISE (in-place)
------------------------------------------------------------
// Step 1: transpose (swap arr[i][j] with arr[j][i])
// Step 2: reverse each row
void rotate90(vector<vector<int>>& mat) {
    int n = mat.size();
    // Transpose
    for(int i=0; i<n; i++)
        for(int j=i+1; j<n; j++)
            swap(mat[i][j], mat[j][i]);
    // Reverse each row
    for(int i=0; i<n; i++)
        reverse(mat[i].begin(), mat[i].end());
}
// [[1,2,3],[4,5,6],[7,8,9]] →
// [[7,4,1],[8,5,2],[9,6,3]]

// Counter-clockwise: reverse each row first, then transpose.


------------------------------------------------------------
E2. SPIRAL ORDER TRAVERSAL
------------------------------------------------------------
vector<int> spiralOrder(vector<vector<int>>& mat) {
    vector<int> res;
    int top=0, bottom=mat.size()-1, left=0, right=mat[0].size()-1;
    while(top<=bottom && left<=right) {
        for(int i=left; i<=right; i++) res.push_back(mat[top][i]); top++;
        for(int i=top; i<=bottom; i++) res.push_back(mat[i][right]); right--;
        if(top<=bottom) {
```

```
            for(int i=right; i>=left; i--) res.push_back(mat[bottom][i]); bottom--;
        }
        if(left<=right) {
            for(int i=bottom; i>=top; i--) res.push_back(mat[i][left]); left++;
        }
    }
    return res;
}


------------------------------------------------------------
E3. SET MATRIX ZEROES (if element is 0, set row and col to 0)
------------------------------------------------------------
void setZeroes(vector<vector<int>>& mat) {
    int m=mat.size(), n=mat[0].size();
    bool firstRowZero=false, firstColZero=false;
    for(int j=0; j<n; j++) if(mat[0][j]==0) firstRowZero=true;
    for(int i=0; i<m; i++) if(mat[i][0]==0) firstColZero=true;
    // use first row/col as markers
    for(int i=1; i<m; i++)
        for(int j=1; j<n; j++)
            if(mat[i][j]==0) { mat[i][0]=0; mat[0][j]=0; }
    // zero out based on markers
    for(int i=1; i<m; i++)
        for(int j=1; j<n; j++)
            if(mat[i][0]==0 || mat[0][j]==0) mat[i][j]=0;
    if(firstRowZero) for(int j=0; j<n; j++) mat[0][j]=0;
    if(firstColZero) for(int i=0; i<m; i++) mat[i][0]=0;
}
Time: O(m*n), Space: O(1)


------------------------------------------------------------
E4. MATRIX MULTIPLICATION
------------------------------------------------------------
vector<vector<int>> multiply(vector<vector<int>>& A, vector<vector<int>>& B) {
    int m=A.size(), k=A[0].size(), n=B[0].size();
    vector<vector<int>> C(m, vector<int>(n, 0));
    for(int i=0; i<m; i++)
        for(int j=0; j<n; j++)
            for(int p=0; p<k; p++)
                C[i][j] += A[i][p] * B[p][j];
    return C;
}
Time: O(m*k*n). For A(m×k) * B(k×n) = C(m×n).
Prerequisite: columns of A must equal rows of B.


------------------------------------------------------------
E5. TRANSPOSE OF MATRIX
------------------------------------------------------------
```

```
vector<vector<int>> transpose(vector<vector<int>>& mat) {
    int m=mat.size(), n=mat[0].size();
    vector<vector<int>> res(n, vector<int>(m));
    for(int i=0; i<m; i++)
        for(int j=0; j<n; j++)
            res[j][i] = mat[i][j];
    return res;
}


------------------------------------------------------------
E6. DIAGONAL TRAVERSAL
------------------------------------------------------------
void printDiagonals(vector<vector<int>>& mat) {
    int m=mat.size(), n=mat[0].size();
    // top-right diagonals (sum of indices = constant)
    for(int d=0; d<m+n-1; d++) {
        for(int i=0; i<m; i++) {
            int j = d-i;
            if(j>=0 && j<n) cout << mat[i][j] << " ";
        }
        cout << "\n";
    }
}


------------------------------------------------------------
E7. FLOOD FILL — DFS ON MATRIX (like paint bucket tool)
------------------------------------------------------------
void dfs(vector<vector<int>>& img, int r, int c, int oldColor, int newColor) {
    if(r<0 || r>=img.size() || c<0 || c>=img[0].size()) return;
    if(img[r][c] != oldColor) return;
    img[r][c] = newColor;
    dfs(img, r+1, c, oldColor, newColor);
    dfs(img, r-1, c, oldColor, newColor);
    dfs(img, r, c+1, oldColor, newColor);
    dfs(img, r, c-1, oldColor, newColor);
}
vector<vector<int>> floodFill(vector<vector<int>>& img, int sr, int sc, int color) {
    int oldColor = img[sr][sc];
    if(oldColor != color) dfs(img, sr, sc, oldColor, color);
    return img;
}


================================================================
PART F — STRING PROBLEMS
================================================================

------------------------------------------------------------
F1. CHECK ANAGRAM
```

```
------------------------------------------------------------
bool isAnagram(string s, string t) {
    if(s.size()!=t.size()) return false;
    int freq[26]={};
    for(char c:s) freq[c-'a']++;
    for(char c:t) freq[c-'a']--;
    for(int x:freq) if(x!=0) return false;
    return true;
}


------------------------------------------------------------
F2. LONGEST COMMON PREFIX
------------------------------------------------------------
string longestCommonPrefix(vector<string>& strs) {
    if(strs.empty()) return "";
    string prefix = strs[0];
    for(int i=1; i<strs.size(); i++)
        while(strs[i].find(prefix) != 0)
            prefix = prefix.substr(0, prefix.size()-1);
    return prefix;
}


------------------------------------------------------------
F3. REVERSE WORDS IN A STRING
------------------------------------------------------------
string reverseWords(string s) {
    istringstream iss(s);
    string word, result;
    vector<string> words;
    while(iss >> word) words.push_back(word);
    for(int i=words.size()-1; i>=0; i--) {
        result += words[i];
        if(i>0) result += " ";
    }
    return result;
}
// "the sky is blue" → "blue is sky the"


------------------------------------------------------------
F4. COUNT OCCURRENCES OF CHARACTER
------------------------------------------------------------
int countChar(string s, char c) {
    return count(s.begin(), s.end(), c);
}


------------------------------------------------------------
F5. STRING TO INTEGER (atoi) — common coding question
```

```
------------------------------------------------------------
int myAtoi(string s) {
    int i=0, sign=1;
    long result=0;
    while(i<s.size() && s[i]==' ') i++;  // skip spaces
    if(i<s.size() && (s[i]=='+'||s[i]=='-'))
        sign = (s[i++]=='+') ? 1 : -1;
    while(i<s.size() && isdigit(s[i])) {
        result = result*10 + (s[i++]-'0');
        if(result*sign > INT_MAX) return INT_MAX;
        if(result*sign < INT_MIN) return INT_MIN;
    }
    return result*sign;
}


================================================================
PART G — TREES & GRAPHS
================================================================

------------------------------------------------------------
G1. BINARY TREE — ALL TRAVERSALS
------------------------------------------------------------
// Inorder: Left → Root → Right (gives SORTED output for BST)
void inorder(TreeNode* root, vector<int>& res) {
    if(!root) return;
    inorder(root->left, res);
    res.push_back(root->val);
    inorder(root->right, res);
}

// Preorder: Root → Left → Right
void preorder(TreeNode* root, vector<int>& res) {
    if(!root) return;
    res.push_back(root->val);
    preorder(root->left, res);
    preorder(root->right, res);
}

// Postorder: Left → Right → Root
void postorder(TreeNode* root, vector<int>& res) {
    if(!root) return;
    postorder(root->left, res);
    postorder(root->right, res);
    res.push_back(root->val);
}

TRICK: Inorder of BST gives sorted array.
       Preorder can reconstruct tree.
       Postorder used in deletion (delete children before parent).

```

```
------------------------------------------------------------
G2. HEIGHT OF BINARY TREE
------------------------------------------------------------
int height(TreeNode* root) {
    if(!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}


------------------------------------------------------------
G3. VALIDATE BINARY SEARCH TREE
------------------------------------------------------------
bool validate(TreeNode* node, long lo, long hi) {
    if(!node) return true;
    if(node->val <= lo || node->val >= hi) return false;
    return validate(node->left, lo, node->val) &&
           validate(node->right, node->val, hi);
}
bool isValidBST(TreeNode* root) {
    return validate(root, LONG_MIN, LONG_MAX);
}


------------------------------------------------------------
G4. LCA OF BINARY TREE (Lowest Common Ancestor)
------------------------------------------------------------
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if(!root || root==p || root==q) return root;
    TreeNode* left  = lca(root->left,  p, q);
    TreeNode* right = lca(root->right, p, q);
    if(left && right) return root;
    return left ? left : right;
}


------------------------------------------------------------
G5. GRAPH — BFS AND DFS
------------------------------------------------------------
// Adjacency list graph
// BFS
void bfs(int start, vector<vector<int>>& adj, int n) {
    vector<bool> visited(n, false);
    queue<int> q;
    q.push(start); visited[start]=true;
    while(!q.empty()) {
        int node = q.front(); q.pop();
        cout << node << " ";
        for(int nb : adj[node])
            if(!visited[nb]) { visited[nb]=true; q.push(nb); }
    }
```

```
}

// DFS (recursive)
void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    cout << node << " ";
    for(int nb : adj[node])
        if(!visited[nb]) dfs(nb, adj, visited);
}


------------------------------------------------------------
G6. DETECT CYCLE IN DIRECTED GRAPH
------------------------------------------------------------
bool dfsCheck(int node, vector<vector<int>>& adj,
              vector<bool>& visited, vector<bool>& recStack) {
    visited[node] = recStack[node] = true;
    for(int nb : adj[node]) {
        if(!visited[nb] && dfsCheck(nb, adj, visited, recStack)) return true;
        if(recStack[nb]) return true;
    }
    recStack[node] = false;
    return false;
}
bool hasCycle(int n, vector<vector<int>>& adj) {
    vector<bool> visited(n,false), recStack(n,false);
    for(int i=0; i<n; i++)
        if(!visited[i] && dfsCheck(i, adj, visited, recStack)) return true;
    return false;
}


------------------------------------------------------------
G7. TOPOLOGICAL SORT (DFS-based)
------------------------------------------------------------
void topoSort(int node, vector<vector<int>>& adj,
              vector<bool>& visited, stack<int>& st) {
    visited[node] = true;
    for(int nb : adj[node])
        if(!visited[nb]) topoSort(nb, adj, visited, st);
    st.push(node);
}
// Print in order by popping stack
// Use case: build order, course prerequisites


------------------------------------------------------------
G8. DYNAMIC PROGRAMMING — COMMON PATTERNS
------------------------------------------------------------

// Fibonacci (DP)
```

```
int fib(int n) {
    if(n<=1) return n;
    int a=0, b=1;
    for(int i=2; i<=n; i++) { int c=a+b; a=b; b=c; }
    return b;
}

// Longest Common Subsequence
int lcs(string s, string t) {
    int m=s.size(), n=t.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for(int i=1; i<=m; i++)
        for(int j=1; j<=n; j++)
            dp[i][j] = (s[i-1]==t[j-1]) ? dp[i-1][j-1]+1
                                        : max(dp[i-1][j], dp[i][j-1]);
    return dp[m][n];
}

// 0/1 Knapsack
int knapsack(vector<int>& w, vector<int>& val, int W) {
    int n=w.size();
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for(int i=1; i<=n; i++)
        for(int j=0; j<=W; j++) {
            dp[i][j] = dp[i-1][j];
            if(j>=w[i-1]) dp[i][j] = max(dp[i][j], dp[i-1][j-w[i-1]]+val[i-1]);
        }
    return dp[n][W];
}

// Coin Change (minimum coins)
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount+1, INT_MAX);
    dp[0]=0;
    for(int i=1; i<=amount; i++)
        for(int c : coins)
            if(c<=i && dp[i-c]!=INT_MAX)
                dp[i] = min(dp[i], dp[i-c]+1);
    return dp[amount]==INT_MAX ? -1 : dp[amount];
}


================================================================
PART H — MCQ TRICKS & APTITUDE SHORTCUTS
================================================================

------------------------------------------------------------
H1. C/C++ OUTPUT TRAPS — MCQ SHORTCUTS
------------------------------------------------------------

TRAP 1: Operator precedence
```

```
  int a=2, b=3, c=4;
  printf("%d", a+b*c);    // OUTPUT: 14 (not 20)
  * before + — always multiply first unless parentheses.

TRAP 2: sizeof
  printf("%d", sizeof(char));       // 1
  printf("%d", sizeof(int));        // 4 (on 32/64-bit)
  printf("%d", sizeof(double));     // 8
  printf("%d", sizeof(int*));       // 8 on 64-bit, 4 on 32-bit
  printf("%d", sizeof("hello"));    // 6 (5 chars + null terminator)
  TRICK: sizeof(array) / sizeof(array[0]) = number of elements

TRAP 3: Integer overflow
  int x = 2147483647;  // INT_MAX
  x++;                 // undefined behavior — wraps to -2147483648
  USE: long long for safety

TRAP 4: Char arithmetic
  char c = 'A';
  printf("%d", c+1);   // OUTPUT: 66 ('A'=65)
  printf("%c", c+1);   // OUTPUT: 'B'
  TRICK: 'A'=65, 'a'=97, '0'=48

TRAP 5: Array out of bounds
  int arr[5]; arr[5]=10;  // UB — no runtime error guaranteed in C++
  This is a common trick question — say "undefined behavior"

TRAP 6: Ternary operator precedence
  int x=5;
  printf("%d", x>3 ? x=1 : x=2);  // OUTPUT: 1 (condition true, x=1)

TRAP 7: Post vs pre increment return
  int a=5;
  int b = a++;   // b=5, a=6 (post: return THEN increment)
  int c = ++a;   // a=7, c=7 (pre: increment THEN return)

TRAP 8: Bitwise operators
  5 & 3 = 1   (0101 & 0011 = 0001)
  5 | 3 = 7   (0101 | 0011 = 0111)
  5 ^ 3 = 6   (0101 ^ 0011 = 0110)
  ~5    = -6  (two's complement: flip all bits)
  5 << 1 = 10 (left shift = multiply by 2)
  5 >> 1 = 2  (right shift = divide by 2, truncate)
  TRICK: n & (n-1) removes the lowest set bit (check if power of 2: n&(n-1)==0)
  TRICK: n ^ n = 0 (XOR with self = 0), n ^ 0 = n

TRAP 9: String and null terminator
  char s[] = "abc";  // sizeof(s)=4 (includes '\0'), strlen(s)=3
  char* p = "abc";   // pointer to string literal — DO NOT MODIFY
  TRICK: strlen counts until '\0', sizeof includes '\0'
```

TRAP 10: Struct padding (MEMORIZE this)
  struct { char a; int b; } → size = 8 (1+3pad+4)
  struct { int a; char b; } → size = 8 (4+1+3pad)
  struct { int a; char b; char c; } → size = 8 (4+1+1+2pad)
  struct { char a; char b; int c; } → size = 8 (1+1+2pad+4)
  RULE: total size = multiple of largest member's size

H2. MATHEMATICS / APTITUDE SHORTCUTS

PERCENTAGE TRICKS:
  x% of y = y% of x (useful: 18% of 50 = 50% of 18 = 9)
  Increase by x% then decrease by x% = net LOSS of ($x^2/100$)%
    (increase 10%, decrease 10% → net -1%)
  A is x% more than B → B is [x/(100+x)]*100 % LESS than A
  If price increases by x%, consumption to keep cost same: x/(100+x)*100 % decrease

RATIO & PROPORTION:
  If A:B = 2:3, total 100 → A=40, B=60
  Compound ratio: (a:b) and (c:d) → ac:bd

TIME & WORK:
  A does in m days, B does in n days → together = mn/(m+n) days
  If A is twice as fast as B → A takes half the time of B
  Work done = (1/days) per day. Add rates when working together.

TIME SPEED DISTANCE:
  Distance = Speed × Time
  Average speed for equal distances = 2xy/(x+y) (NOT simple average)
  If speed increases by x%, time decreases by x/(100+x)*100 %

PROFIT & LOSS:
  Profit % = (Profit / CP) × 100
  SP = CP × (1 + P/100)
  Marked Price discount: SP = MP × (1 - D/100)
  Successive discount d1% and d2% = (d1+d2 - d1*d2/100)%

SIMPLE vs COMPOUND INTEREST:
  SI = (P × R × T) / 100
  CI = P × (1 + R/100)^T - P
  Difference for 2 years = P × $(R/100)^2$

NUMBER TRICKS:
  Sum of first n natural numbers = n(n+1)/2
  Sum of squares = n(n+1)(2n+1)/6
  Sum of cubes = $[n(n+1)/2]^2$
  Number of divisors of n: factorize, add 1 to each exponent, multiply
    e.g. 12 = $2^2 \times 3^1$ → (2+1)(1+1) = 6 divisors

DIVISIBILITY RULES (MCQ speed tricks):
  ÷2: last digit even
  ÷3: sum of digits divisible by 3
  ÷4: last 2 digits divisible by 4
  ÷5: last digit 0 or 5
  ÷6: divisible by 2 AND 3
  ÷7: double last digit, subtract from rest, check if div by 7
  ÷8: last 3 digits divisible by 8
  ÷9: sum of digits divisible by 9
  ÷11: alternating sum divisible by 11

PROBABILITY:
  P(A or B) = P(A) + P(B) - P(A and B)
  P(A and B) independent = P(A) × P(B)
  P(not A) = 1 - P(A)
  Coin: P(head) = 1/2. Two heads = 1/4. At least one head = 3/4.
  Dice: P(any number) = 1/6. Sum=7 most likely (6 ways out of 36).

PERMUTATION & COMBINATION:
  nPr = n! / (n-r)!   (ORDER matters)
  nCr = n! / (r!(n-r)!) (ORDER doesn't matter)
  nCr = nC(n-r)
  Trick: nC0 = nCn = 1, nC1 = n
  Arrangements of "MISSISSIPPI" = 11! / (4!4!2!) (repeated letters)

SERIES / SEQUENCES:
  AP: a, a+d, a+2d... sum = n/2 × (2a+(n-1)d)
  GP: a, ar, ar²... sum = a(rⁿ-1)/(r-1) for r≠1
  Identify pattern: look at differences, then differences of differences

---

H3. LOGICAL REASONING SHORTCUTS

DIRECTION PROBLEMS:
  Start facing North. Turn right = face East. Turn right again = South.
  Left turn from North = West.
  TRICK: draw the compass. N/S/E/W. Right = clockwise.

BLOOD RELATIONS:
  "A is the son of B's father's only son" — draw the family tree.
  Don't trust words, draw every time.

CODING-DECODING:
  Letter positions: A=1, B=2... Z=26
  Reverse: A=26, B=25...
  Shift by 1: A→B, B→C... (Caesar cipher)
  Opposite: A↔Z, B↔Y, C↔X...

NUMBER SERIES — IDENTIFY FAST:

  Differences constant → AP
  Ratios constant → GP
  Differences increasing by constant → quadratic sequence
  Primes: 2,3,5,7,11,13,17,19,23...
  Squares: 1,4,9,16,25,36...
  Cubes: 1,8,27,64,125...
  Fibonacci: 1,1,2,3,5,8,13,21...

H4. BIG-O COMPLEXITY QUICK REFERENCE (MCQ staple)

O(1)   — array index, hashmap lookup, push/pop stack
O(log n) — binary search, BST lookup, heap push/pop
O(n)   — linear search, single pass, linked list traversal
O(n log n)— merge sort, heap sort, good sorting
O(n²)   — bubble/selection/insertion sort, nested loops
O(2ⁿ)   — all subsets, brute force exponential
O(n!)   — permutations, traveling salesman brute force

TRICK: nested for loops over same array = O(n²)
       recursion halving input = O(log n)
       recursion on all elements with O(n) per level = O(n log n)

==========
PART I — COMMONLY ASKED THEORY (SHORT ANSWERS)
==========

Q. What is Big Endian vs Little Endian?
A. Big Endian: MSB stored at lowest address. Network byte order.
   Little Endian: LSB stored at lowest address. x86 default.
   int 0x12345678 at address 0x100:
   Big: 0x100=12, 0x101=34, 0x102=56, 0x103=78
   Little: 0x100=78, 0x101=56, 0x102=34, 0x103=12
   Check in code: int x=1; char* p=(char*)&x; if(*p==1) → little endian.

Q. What is Two's Complement?
A. How negative numbers are stored. Flip all bits + 1.
   5 = 00000101
   -5 = 11111010 + 1 = 11111011
   Why: addition works the same way. 5 + (-5) = 0 with natural overflow.
   Range for 8-bit: -128 to 127

Q. What is static variable?
A. In function: value persists between calls (initialized once).
   In class: shared across all objects (ONE copy for whole class).
   At file scope: internal linkage (not visible outside translation unit).
   void count() { static int c=0; c++; printf("%d",c); }
   count(); count(); count(); // prints 1 2 3

Q. What is inline function?
A. Hint to compiler to insert function body at call site (avoid call overhead).
   inline int square(int x) { return x*x; }
   Modern compilers decide themselves — inline keyword is mostly advisory now.
   DON'T inline: recursive functions, large functions.

Q. Difference between #define and const in C++?
A. #define: preprocessor text replacement, no type, no scope
   const: type-safe, has scope, debugger can see it
   Prefer const (or constexpr) over #define in C++.

Q. What is constexpr?
A. Value computed at compile time. More powerful than const.
   constexpr int SIZE = 10;   // computed at compile time
   constexpr int square(int x) { return x*x; }
   constexpr int s = square(5);   // 25, computed at compile time

Q. What is extern?
A. Declares a variable defined in another translation unit.
   // file1.cpp: int globalVar = 5;
   // file2.cpp: extern int globalVar; // just a declaration

Q. What is the difference between ++i and i++ performance?
A. For primitives: no difference (compiler optimizes).
   For iterators/objects: ++i avoids creating a temporary copy.
   Use ++i by default in loops: for(auto it=v.begin(); it!=v.end(); ++it)

Q. What is a virtual table (vtable) size overhead?
A. Each polymorphic class gets one vtable. Each object gets one vptr (8 bytes).
   Only the pointer per object — not per virtual function per object.
   class Base { virtual void f(); };
   sizeof(Base) includes 8 bytes for vptr.

Q. What is the one definition rule (ODR)?
A. Each variable/function must be defined exactly once across the program.
   Multiple declarations fine, but one definition.
   Inline functions and templates are exceptions (can be defined in headers).

Q. What is name mangling?
A. C++ compiler encodes function signature into the symbol name to support
   overloading. f(int) → _Z1fi, f(double) → _Z1fd
   extern "C" disables mangling (needed when calling C++ from C).

Q. What does volatile keyword do?
A. Tells compiler: don't optimize this variable. Read it fresh every time.
   Use for: hardware registers, variables modified by signal handlers,
   shared variables without mutex (though atomic is better now).
   YOUR ANSWER: "I could use volatile for memory-mapped device registers
   in kernel-space code."

```
================================================================
PART J — 3 QUESTIONS TO ASK INTERVIEWER (refined)
================================================================

1. "For the Windows driver work — is the team using KMDF or UMDF,
   and is most of the codebase in C or modern C++? I want to know
   where to focus my Windows ramp-up."

2. "What does the first 3 months look like — is there a buddy system
   or will I be diving into the codebase directly? And are there any
   existing Linux/macOS modules I could contribute to while ramping
   up on Windows?"

3. "Is this role contributing to an existing product line or is there
   a new module being designed from scratch? I ask because I'm most
   effective when I can own the design end-to-end."


================================================================
PART K — QUICK REVISION — READ ON TRAIN TOMORROW
================================================================

SORTING IN ONE LINE:
  Bubble/Selection/Insertion = O(n²). Simple. Cache-friendly.
  Merge = O(n log n) stable. Needs O(n) space.
  Quick = O(n log n) avg. In-place. Worst O(n²). std::sort uses this.
  Heap  = O(n log n) always. In-place. Not stable.

BINARY SEARCH: sorted array, lo+hi/2, check mid, move lo or hi.

ROTATE ARRAY: reverse trick. Right k: reverse all, reverse [0..k-1], reverse [k..n-1].

MATRIX ROTATE 90°: transpose then reverse each row.

FLOYD'S CYCLE: slow +1, fast +2. If they meet → cycle.

KADANE'S: currSum = max(num, currSum+num). Track maxSum.

TWO SUM: hashmap, store index, look for complement.

STACK use: parentheses, next greater element, undo operations.
QUEUE use: BFS, level order, print in FIFO order.

VTABLE: pointer per object → table of function ptrs per class.
RAII: destructor = free. Always. Even on exception.
RULE OF 5: define all 5 or none when managing raw resources.
UNIQUE_PTR: one owner. SHARED_PTR: ref count. WEAK_PTR: no ownership.

DEADLOCK: 4 conditions — mutual exclusion, hold+wait, no preemption, circular wait.
MUTEX vs SEMAPHORE: mutex has ownership. Semaphore doesn't.
```

```
WINDOWS BRIDGE:
  fanotify        → minifilter callback
  inotify         → ReadDirectoryChangesW
  Unix socket     → Named pipe
  Linux .ko       → KMDF driver
  /proc /sys      → Registry / DeviceIoControl
  SIGKILL         → TerminateProcess()

================================================================
BEST OF LUCK TOMORROW — 2 PM
BUILD ON YOUR ACTUAL EXPERIENCE. EVERY ANSWER LINKS TO YOUR WORK.
================================================================
```